

PROJEKT-DOKUMENTATION:

FUHRPARKVERWALTUNG

Analyse der Architektur, Design Patterns und verwendeten Technologien (Stand: 5. Ausbildungsmonat)

Architektur: MVC

Persistenz: JPA / Hibernate

Pattern: Factory & DI

1. Einleitung & Projektübersicht

Dieses Dokument fasst die softwarearchitektonischen Entscheidungen, die implementierten Entwurfsmuster (Design Patterns) sowie die technologische Infrastruktur der Applikation **Fuhrparkverwaltung** zusammen. Das System dient der Erfassung, Verwaltung und relationalen Persistierung von Fahrzeugen (PKW, LKW, Motorrad) und Mitarbeitern sowie deren gegenseitiger Zuordnung.

Der Code wurde im 5. Monat der Ausbildung zum Applikationsentwickler auf einem Niveau umgesetzt, das durch die strikte Einhaltung moderner Software-Engineering-Prinzipien und die saubere Integration eines objektrelationalen Mappers (ORM) weit über die typischen Grundlagen hinausgeht.

2. Verzeichnis- und Klassenstruktur

Das Projekt ist konsequent nach dem **Model-View-Controller (MVC)** Muster strukturiert, um eine klare Trennung der Zuständigkeiten (Separation of Concerns) zu gewährleisten:

```
src/ └─ Controller/ | └─ ParkManager.java <-- Zentrale Geschäftslogik & Steuerung
└─ Model/ | └─ Employee.java <-- Entität für Mitarbeiterdaten | └─ Vehicle.java <--
Abstrakte Basisklasse für Fahrzeuge | └─ Pkw.java <-- Spezifische PKW-Entität | └─
Lkw.java <-- Spezifische LKW-Entität | └─ Motorcycle.java <-- Spezifische Motorrad-
Entität | └─ VehicleFactory.java <-- Fabrikmuster für Objekterzeugung └─ View/ | └─
Menu.java <-- Konsolenbasierte Benutzeroberfläche └─ Main.java <--
Applikationseinstieg & DI-Verdrahtung
```

3. Implementierte Design Patterns

3.1 Erzeugungsmuster (Creational Patterns)

- **Factory Method Pattern (Fabrikmuster):** Die Erzeugung der konkreten Fahrzeug-Subklassen (Pkw, Lkw, Motorcycle) erfolgt nicht direkt im Controller, sondern ist in die Klasse `VehicleFactory` ausgelagert. Dies entkoppelt die Anwendungslogik von den spezifischen Konstruktoren der Unterklassen und erleichtert spätere Erweiterungen um neue Fahrzeugtypen.
- **Dependency Injection (DI / Abhängigkeitsinjektion):** Klassen instanziierten ihre benötigten Werkzeuge und Services nicht selbst mittels des `new`-Operators. Stattdessen werden sie von außen übergeben. In der `Main.java` wird dies über die **Constructor Injection** umgesetzt (z. B. Übergabe des `EntityManager`

an den `ParkManager`). Das Menü nutzt zudem **Method Injection** in der `showMenu`-Methode, um temporäre Abhängigkeiten dynamisch zu erhalten. Dies erhöht die Testbarkeit und Flexibilität des gesamten Codes enorm.

3.2 Struktur- und Verhaltensmuster

- **Domain Model Pattern:** Die Klassen im `Model`-Paket bilden die reale Domäne der Fuhrparkverwaltung ab. Sie enthalten nicht nur Daten, sondern spiegeln die logischen Zusammenhänge und Geschäftsregeln des realen Gegenstücks wider.
- **Separation of Concerns (Trennung der Zuständigkeiten):** Jede Komponente hat genau eine Aufgabe. Die `Main` verdrahtet die Anwendung, das `Menu` kümmert sich um die Interaktion mit dem Anwender, der `ParkManager` steuert die Geschäftsabläufe und das `Model` hält den Zustand.

4. Technologische Infrastruktur & Datenbank-Patterns

Das Projekt nutzt **JPA (Java Persistence API)** mit **Hibernate** als Implementierung (ORM-Provider), um die Java-Objektwelt transparent auf eine relationale **PostgreSQL**-Datenbank abzubilden.

Vererbungsstrategie: Class Table Inheritance (JOINED)

Durch die Annotation `@Inheritance(strategy = InheritanceType.JOINED)` auf der abstrakten Basisklasse `Vehicle` wird ein hochgradig normalisiertes Datenbankschema erzeugt. Die gemeinsamen Attribute liegen in der Tabelle `vehicle`, während spezifische Attribute in separaten Tabellen (`pkw`, `lkw`, `motorcycle`) verwaltet werden. Die Verknüpfung erfolgt relational über Primärschlüssel-Fremdschlüssel-Beziehungen (`@PrimaryKeyJoinColumn`).

Im Hintergrund genutzte Enterprise-Patterns:

Pattern / Konzept	Bedeutung und Funktion im Projekt
Object-Relational Mapping (ORM)	Automatisches Übersetzen von Java-Objekten in relationale Tabellenstrukturen und SQL-Statements, gesteuert über Metadaten (Annotationen wie @Entity, @Table).
Data Mapper Pattern	Strikte Trennung zwischen Daten und Persistenzlogik. Die Entitäten sind reine Domänenobjekte und wissen nichts von der Datenbank. Die Speicherung wird exklusiv über den EntityManager abgewickelt.
Unit of Work / Identity Map	Der EntityManager fungiert als zentraler Kontext. Er trackt Änderungen an Objekten innerhalb einer Transaktion und cacht bereits geladene Entitäten im First-Level-Cache, um redundante Datenbankzugriffe zu verhindern.
Element Collection	Über @ElementCollection werden einfache Datentyp-Listen (wie die licenseList im Employee) ohne die Komplexität einer eigenen Entitätsklasse in eine verknüpfte Hilfstabelle (employee_licenses) ausgelagert.
Repository / JPQL	Abfragen erfolgen plattformunabhängig und objektorientiert über die <i>Java Persistence Query Language</i> (z. B. SELECT v FROM Vehicle v), anstatt hartes, datenbankspezifisches SQL zu schreiben.

5. Bewertung & Zukünftige Optimierungspotenziale

Der aktuelle Softwarestand zeigt ein tiefes Verständnis für komponentenbasierte Softwareentwicklung und relationale Datenbankankbindung. Für die fortlaufende Entwicklung wurden folgende Optimierungsschritte im Rahmen der Ausbildung definiert:

- 1. Vollständiges Unit-Testing (Nächster Schritt):** Abdeckung der Kernkomponenten (insb. der VehicleFactory und der Business-Logik des Controllers) durch automatisierte Unit-Tests unter Verwendung von JUnit 5 und Mockito, um den Zustand isoliert und ohne lebende Datenbankverbindung prüfen zu können.
- 2. Strikte UI-Eingabetrennung:** Sukzessives Herauslösen des Scanner-Objekts aus dem ParkManager. Der Controller sollte perspektivisch reine Datenstrukturen oder DTOs (Data Transfer Objects) aus der View entgegennehmen, um komplett frei von Konsolen-Eingabelogik zu werden.
- 3. Erweiterte Validierung:** Ausbau der Absicherung von numerischen Konsoleneingaben (z.B. Kilometerstand, Tankfüllung) analog zur bereits implementierten Ganzzahl-Validierung im Hauptmenü.